

JPA & Hibernate: Entity Mapping & Relationships

A useful mental model:

```
Java Object
  ↓
JPA mapping
  ↓
Hibernate
  ↓
SQL
  ↓
Relational Database
```

1. JPA vs Hibernate

First, don't treat them as the same thing.

JPA

Jakarta Persistence (JPA) is a specification/API defining how Java applications map objects to relational databases.

It provides concepts such as:

```
@Entity
@Id
@OneToMany
@ManyToOne
@OneToOne
@ManyToMany
```

Hibernate

Hibernate is an implementation of the JPA specification.

```
JPA
  ↓
Specification

Hibernate
  ↓
Implementation
  ↓
Generates SQL
  ↓
Communicates with DB
```

So an interview answer:

JPA defines the ORM standard and APIs, while Hibernate is a popular implementation of that standard.

2. What Is ORM?

ORM = **Object-Relational Mapping**.

It maps:

Java		Database
Class	→	Table
Object	→	Row
Field	→	Column
Relationship	→	Foreign Key

Example Java:

```
public class Payment {  
  
    private Long id;  
    private BigDecimal amount;  
    private String status;  
}
```

Database:

```
CREATE TABLE payment (  
    id BIGINT PRIMARY KEY,  
    amount DECIMAL(10,2),  
    status VARCHAR(20)  
);
```

Hibernate manages the conversion between the two representations.

3. Basic Entity

A JPA entity is normally annotated with:

```
@Entity  
@Table(name = "payment")  
public class Payment {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private BigDecimal amount;  
  
    private String status;  
}
```

Hibernate maps this approximately to:

```
Payment
  ↓
payment table

id      → id
amount  → amount
status  → status
```

4. @Entity

```
@Entity
public class Payment {
}
```

tells JPA:

This class should be persisted as an entity.

An entity needs an identifier.

```
@Id
private Long id;
```

5. @Table

You can specify the database table:

```
@Entity
@Table(name = "payments")
public class Payment {
}
```

Without it, the ORM provider may derive a table name from the entity name according to its naming strategy.

6. @Id

Every entity must have an identifier.

```
@Id
private Long id;
```

This maps to the primary-key concept.

Example:

```
CREATE TABLE payment (  
  id BIGINT PRIMARY KEY  
);
```

7. ID Generation

Common strategy:

```
@Id  
@GeneratedValue(strategy = GenerationType.IDENTITY)  
private Long id;
```

Other strategies include:

```
IDENTITY  
SEQUENCE  
TABLE  
AUTO
```

Important interview distinction

IDENTITY often relies on the database generating the value through an identity/auto-increment mechanism.

SEQUENCE uses a database sequence where supported.

Example:

```
@Id  
@GeneratedValue(strategy = GenerationType.SEQUENCE)  
private Long id;
```

With some databases, sequence-based generation can allow Hibernate to optimize identifier allocation more effectively than identity columns.

8. Column Mapping

You can customize columns:

```
@Column(name = "payment_amount", nullable = false)  
private BigDecimal amount;
```

Useful attributes include:

```
name  
nullable  
unique  
length  
precision  
scale  
insertable  
updatable
```

Example:

```
@Column(  
    name = "amount",  
    nullable = false,  
    precision = 12,  
    scale = 2  
)  
private BigDecimal amount;
```

9. Entity Relationships

The major relationships are:

One-to-One
One-to-Many
Many-to-One
Many-to-Many

Think in database terms:

Table A
↓
Foreign Key
↓
Table B

10. @ManyToOne

This is probably the **most important relationship to understand well**.

Suppose:

Customer
|
├─ Payment
├─ Payment
└─ Payment

Many payments belong to one customer.

Database:

```
customer  
-----  
id  
  
payment  
-----  
id  
customer_id ← FK  
amount
```

Entity:

```
@Entity
public class Payment {

    @Id
    @GeneratedValue
    private Long id;

    private BigDecimal amount;

    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

The important part:

```
@JoinColumn(name = "customer_id")
```

means the `payment` table contains:

```
customer_id
```

as the foreign key.

11. Why `ManyToOne` Is Usually the Owning Side

Consider:

```
Payment → Customer
```

The payment table contains:

```
customer_id
```

Therefore the `Payment` entity owns the relationship from the relational perspective.

```
Payment
  ↓
customer_id
  ↓
Customer
```

This is why you'll commonly see:

```
@ManyToOne
@JoinColumn(name = "customer_id")
```

12. `@OneToMany`

Now reverse the relationship.

One customer has many payments:

```
@Entity
public class Customer {

    @Id
    private Long id;

    @OneToMany(mappedBy = "customer")
    private List<Payment> payments;
}
```

Notice:

```
mappedBy = "customer"
```

This means:

The `Payment.customer` field owns the relationship.

It does **not** mean the database has a column called `customer`.

That's an important distinction.

13. Bidirectional Relationship

Now both sides know about each other.

```
@Entity
public class Customer {

    @OneToMany(mappedBy = "customer")
    private List<Payment> payments;
}
```

and:

```
@Entity
public class Payment {

    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

Conceptually:

```
Customer
  ↑
  |
payments
  |
  ↓
Payment
  ↑
  |
customer
```

This is a **bidirectional relationship**.

14. `mappedBy` — Very Important

Suppose:

```
@OneToMany(mappedBy = "customer")
private List<Payment> payments;
```

`mappedBy = "customer"` refers to the **Java field name**, not the database column.

This:

```
mappedBy = "customer"
```

corresponds to:

```
private Customer customer;
```

inside `Payment`.

Not:

```
customer_id
```

This is a common interview mistake.

15. Owning Side

JPA relationships have an **owning side**.

The owning side is generally the side responsible for managing the foreign-key relationship.

Example:

```
@ManyToOne
@JoinColumn(name = "customer_id")
private Customer customer;
```

is the owning side.

The inverse side:

```
@OneToMany(mappedBy = "customer")
private List<Payment> payments;
```

doesn't independently manage the foreign-key column.

16. Keeping Both Sides in Sync

Suppose you have:

```
Customer customer = new Customer();
Payment payment = new Payment();
```

A good domain model often provides helper methods:

```
public void addPayment(Payment payment) {
    payments.add(payment);
    payment.setCustomer(this);
}
```

Why?

Because in a bidirectional relationship:

```
Customer.payments
```

and:

```
Payment.customer
```

are both in-memory object references.

You want both sides consistent.

17. @OneToOne

Example:

```
User
|
└─ UserProfile
```

One user has one profile.

```
@Entity
public class User {

    @Id
    private Long id;

    @OneToOne
    @JoinColumn(name = "profile_id")
    private UserProfile profile;
}
```

Database:

```
user
-----
id
profile_id ← FK

user_profile
-----
id
```

...

But one-to-one mappings require careful database constraints.

For true one-to-one semantics, the foreign key generally needs to be unique.

18. @ManyToMany

Example:

```
Student
  ↓
Courses
```

One student can take multiple courses.

One course can have multiple students.

Relational databases typically need a **join table**:

```
student
  |
  ↓
student_course
  ↑
  |
course
```

Example:

```
@Entity
public class Student {

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses;
}
```

Join table:

```
student_course
-----
student_id
course_id
```

19. Why Set Is Often Used for Many-to-Many

Example:

```
private Set<Course> courses;
```

rather than:

```
private List<Course> courses;
```

A `Set` naturally represents uniqueness.

But don't say:

"@ManyToMany must always use Set."

It doesn't.

The choice depends on domain semantics and ordering requirements.

20. Relationship Summary

Relationship	Example	Typical DB representation
@OneToOne	User → Profile	FK + uniqueness
@OneToMany	Customer → Payments	FK on many side
@ManyToOne	Payment → Customer	FK
@ManyToMany	Student ↔ Course	Join table

The most commonly used relationship in typical business systems is often:

```
@ManyToOne
```

because many records reference one parent.

21. Fetch Type

This is extremely important.

JPA relationships can generally be:

```
EAGER  
LAZY
```

EAGER

Associated data is loaded immediately according to the provider's loading strategy.

LAZY

Associated data is loaded when accessed, typically through Hibernate's proxy/enhancement mechanisms.

Example:

```
@OneToMany(fetch = FetchType.LAZY)
private List<Payment> payments;
```

22. Why LAZY Is Usually Preferred for Collections

Suppose:

```
Customer
  ↓
10,000 Payments
```

You query:

```
customerRepository.findById(1L);
```

If payments are unnecessarily loaded immediately:

```
Customer
+
10,000 payment rows
```

could be expensive.

With lazy loading:

```
Customer
  ↓
Payments loaded only when needed
```

This avoids unnecessary work.

23. Default Fetch Types — Know the Nuance

A useful interview fact:

```
@ManyToOne → EAGER by default
@OneToOne → EAGER by default

@OneToMany → LAZY by default
@ManyToMany → LAZY by default
```

This is one reason teams often explicitly configure fetch behavior for important relationships rather than relying blindly on defaults.

24. The N+1 Query Problem

This is a **very important Hibernate interview topic**.

Suppose you query:

```
List<Customer> customers =  
    customerRepository.findAll();
```

Then iterate:

```
for (Customer customer : customers) {  
    customer.getPayments().size();  
}
```

Potentially:

```
1 query  
↓  
Load customers  
  
N queries  
↓  
Load payments for each customer
```

So:

```
1 + N queries
```

This is the **N+1 query problem**.

Example:

```
Query 1:  
SELECT * FROM customer;  
  
Query 2:  
SELECT * FROM payment WHERE customer_id = 1;  
  
Query 3:  
SELECT * FROM payment WHERE customer_id = 2;  
  
Query 4:  
SELECT * FROM payment WHERE customer_id = 3;  
...
```

This can seriously hurt performance.

25. How Do You Solve N+1?

Several techniques exist.

Fetch join

JPQL:

```
@Query("""  
    SELECT c  
    FROM Customer c  
    JOIN FETCH c.payments  
""")  
List<Customer> findCustomersWithPayments();
```

Conceptually:

```
Customer + Payment
      ↓
single/fewer SQL operations
```

But fetch joins need careful use, especially with collections and pagination.

Entity graphs

```
@EntityGraph(attributePaths = "payments")
List<Customer> findAll();
```

Entity graphs can specify which associations should be fetched for a particular operation.

Batch fetching

Hibernate can batch loading of lazy associations rather than issuing one query per parent.

26. LazyInitializationException

Classic Hibernate question.

Suppose:

```
Customer customer =
    repository.findById(id).orElseThrow();
```

The transaction/session ends.

Later:

```
customer.getPayments();
```

Hibernate may try to initialize a lazy association after the persistence context is no longer available.

You can get:

```
LazyInitializationException
```

The key lesson:

*Don't blindly solve this by making every relationship **EAGER**.*

Instead, define the fetch requirements at the use-case/query boundary.

27. Cascade

Cascade controls whether certain entity operations propagate from one entity to associated entities.

Example:

```
@OneToMany(  
    mappedBy = "customer",  
    cascade = CascadeType.ALL  
)  
private List<Payment> payments;
```

Then operations such as:

```
PERSIST  
MERGE  
REMOVE  
REFRESH  
DETACH
```

can be cascaded according to the configuration.

28. Cascade Is Not Fetching

Another common trap.

```
Cascade  
    → How entity operations propagate  
  
FetchType  
    → When associations are loaded
```

They solve completely different problems.

For example:

```
cascade = CascadeType.ALL  
fetch = FetchType.LAZY
```

is perfectly valid.

It means:

```
Operations propagate  
+  
relationship remains lazily loaded
```

29. orphanRemoval

Example:

```
@OneToMany(  
    mappedBy = "customer",  
    orphanRemoval = true  
)  
private List<Payment> payments;
```

If a child entity is removed from the parent's managed collection, orphan removal can cause the child row to be deleted, depending on the entity lifecycle and operation.

Conceptually:

```
Customer
|
├── Payment A
└── Payment B

Remove Payment A from collection
    ↓
Payment A can be deleted
```

Use this carefully.

It is appropriate when the child is conceptually owned by the parent and shouldn't exist independently.

30. Cascade vs `orphanRemoval`

Cascade	<code>orphanRemoval</code>
Propagates entity operations	Deletes removed/orphaned children
Parent operation → child	Child removed from relationship → deletion
<code>CascadeType.PERSIST</code> , <code>REMOVE</code> , etc.	Boolean
Lifecycle propagation	Orphan lifecycle semantics

31. Embedded Objects

Not every value deserves its own entity/table.

Suppose:

```
class Address {
    String city;
    String state;
    String pinCode;
}
```

You can use:

```
@Embeddable
public class Address {

    private String city;
    private String state;
    private String pinCode;
}
```

Then:

```
@Entity
public class Customer {
```

```
@Embedded
private Address address;
}
```

The fields can live in the same table:

```
customer
-----
id
city
state
pin_code
```

This is useful for **value objects** that don't need an independent identity.

32. Entity vs Embeddable

Entity

Has:

```
identity
lifecycle
persistence identity
```

Example:

```
Payment
Customer
Order
```

Embeddable

Represents a value belonging to another entity.

Example:

```
Address
Money
AuditMetadata
```

Conceptually:

```
Entity → independent identity
Embeddable → part of another entity
```

33. DTO vs Entity

Don't expose JPA entities blindly from REST APIs.

For example:

```
@GetMapping("/payments/{id}")
public Payment getPayment(...) {
    ...
}
```

can create problems such as:

- exposing internal fields
- lazy-loading issues
- recursive serialization
- unstable API contracts
- tightly coupling API representation to database model

Prefer:

```
Controller
  ↓
DTO
  ↓
Service
  ↓
Entity
  ↓
Repository
```

Example:

```
public record PaymentResponse(
    Long id,
    BigDecimal amount,
    String status
) {}
```

34. Infinite JSON Recursion

A classic problem with bidirectional mappings.

Suppose:

```
Customer
  → payments
    → customer
      → payments
        → customer
```

Jackson can encounter recursive traversal when serializing entities.

You may see approaches involving:

```
@JsonIgnore
@JsonManagedReference
@JsonBackReference
```

But in production APIs, a cleaner architectural approach is often:

Use DTOs rather than exposing the bidirectional entity graph directly.

35. @JoinColumn vs mappedBy

Memorize this.

@JoinColumn

Specifies the foreign-key column used for the association.

```
@ManyToOne
@JoinColumn(name = "customer_id")
private Customer customer;
```

mappedBy

Specifies the field on the other entity that owns the relationship.

```
@OneToMany(mappedBy = "customer")
private List<Payment> payments;
```

So:

```
@JoinColumn
  ↓
database FK mapping

mappedBy
  ↓
inverse side references owning Java field
```

36. Relationship Example End-to-End

Let's model:

```
Customer
|
| 1
|
| N
|
↓
Payment
```

Customer

```
@Entity
@Table(name = "customer")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
```

```

    @OneToMany(
        mappedBy = "customer",
        fetch = FetchType.LAZY
    )
    private List<Payment> payments = new ArrayList<>();

    public void addPayment(Payment payment) {
        payments.add(payment);
        payment.setCustomer(this);
    }
}

```

Payment

```

@Entity
@Table(name = "payment")
public class Payment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private BigDecimal amount;

    private String status;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id")
    private Customer customer;

    public void setCustomer(Customer customer) {
        this.customer = customer;
    }
}

```

Database:

```

customer
-----
id
name

payment
-----
id
amount
status
customer_id → customer.id

```

The relationship is:

```

Customer 1 ————— N Payment

```

37. What Happens When Hibernate Loads an Entity?

Suppose:

```

Customer customer =
    customerRepository.findById(1L)

```

```
.orElseThrow();
```

Hibernate translates the repository operation into SQL approximately like:

```
SELECT id, name
FROM customer
WHERE id = 1;
```

Because:

```
payments = LAZY
```

Hibernate doesn't necessarily load the payments immediately.

Later:

```
customer.getPayments();
```

may trigger another query:

```
SELECT *
FROM payment
WHERE customer_id = 1;
```

That's the basic idea behind lazy loading.

38. Persistence Context

You should know this alongside entity mapping.

Inside a transaction, JPA maintains a **persistence context**.

Think:

```
Database
  ↑
Persistence Context
  ↑
Java Entities
```

If Hibernate loads:

```
Payment payment = repository.findById(1L);
```

the entity becomes managed.

Changes such as:

```
payment.setStatus("PAID");
```

can be detected through **dirty checking** and synchronized to the database when the persistence context is flushed.

You don't necessarily need:

```
repository.save(payment);
```

for every modification to an already-managed entity.

39. Dirty Checking

Example:

```
@Transactional
public void markPaid(Long id) {

    Payment payment =
        repository.findById(id).orElseThrow();

    payment.setStatus("PAID");
}
```

You don't explicitly execute:

```
UPDATE payment ...
```

Hibernate can detect that the managed entity changed and generate an update during flush.

Conceptually:

```
Load entity
↓
Managed entity
↓
Change field
↓
Hibernate detects change
↓
Flush
↓
UPDATE SQL
```

This is a very common Hibernate interview question.

40. Entity Lifecycle

Know these states:

```
TRANSIENT
  ↓ persist()
MANAGED
  ↓ detach()
DETACHED
  ↓ merge()
MANAGED

MANAGED
  ↓ remove()
REMOVED
```

Transient

New Java object, not associated with persistence context.

```
Payment p = new Payment();
```

Managed

Tracked by the persistence context.

```
entityManager.persist(p);
```

Detached

Was managed but is no longer associated with the current persistence context.

Removed

Marked for deletion.

41. `save()` vs `persist()` vs `merge()`

At a 3-year level, understand the conceptual distinction.

`persist()`

Makes a new entity managed.

```
entityManager.persist(payment);
```

`merge()`

Copies state from a detached entity into a managed entity and returns the managed instance.

```
Payment managed =  
    entityManager.merge(detachedPayment);
```

Important:

The object passed to `merge()` does not become the managed instance simply because you called `merge()`.

The returned object is the one you should use as the managed entity.

42. Common Interview Questions

Entity mapping

1. What is JPA?
2. JPA vs Hibernate?
3. What is ORM?
4. What is an entity?
5. What does `@Entity` do?
6. How does `@Id` work?
7. Difference between `IDENTITY` and `SEQUENCE` ?
8. What is `@Column` ?
9. What is `@JoinColumn` ?
10. What does `mappedBy` mean?

Relationships

11. Explain `OneToOne` .
12. Explain `OneToMany` .
13. Explain `ManyToOne` .
14. Explain `ManyToMany` .
15. What is the owning side?
16. Why is `ManyToOne` commonly the owning side?
17. `mappedBy` vs `@JoinColumn` .
18. Unidirectional vs bidirectional relationship?
19. When would you avoid bidirectional relationships?

Hibernate

20. Lazy vs eager loading?
21. What is N+1?
22. How do you solve N+1?
23. What is `LazyInitializationException` ?
24. What is persistence context?
25. What is dirty checking?
26. Explain entity lifecycle.
27. Cascade vs orphan removal.
28. Cascade vs fetch type.
29. Entity vs DTO?
30. How can bidirectional mapping cause JSON recursion?

43. Common Traps

Trap 1

"Hibernate is JPA."

✗ Incorrect.

JPA = specification

Hibernate = implementation

Trap 2

"mappedBy points to the database column."

✗ Incorrect.

It points to the **Java association field/property** on the owning side.

Trap 3

"LAZY means zero SQL until I access the field."

Too simplistic.

Hibernate may use proxies/enhancement and provider-specific behavior; the important point is that the association is not intended to be fully initialized immediately.

Trap 4

"Just make everything EAGER to solve LazyInitializationException."

✗ Bad approach.

It can create:

```
huge object graphs
+
unnecessary joins/queries
+
N+1 problems
+
poor performance
```

Design fetch plans around use cases.

Trap 5

"Cascade means Hibernate automatically fetches children."

✗ Wrong.

```
Cascade → lifecycle/operation propagation
Fetch → loading strategy
```

44. Interview-Ready Answer

If asked:

“Explain JPA entity relationships.”

A good answer:

JPA provides ORM mappings between Java entities and relational tables. An entity is identified using `@Id`, and relationships are modeled with `@OneToOne`, `@OneToMany`, `@ManyToOne`, and `@ManyToMany`. For example, if many payments belong to one customer, `Payment` typically has a `@ManyToOne` association with a `@JoinColumn` containing the foreign key, while `Customer` can have a `@OneToMany(mappedBy = "customer")` inverse collection. I also need to consider ownership, lazy versus eager loading, cascade behavior, orphan removal, and N+1 query problems. Hibernate implements these JPA mappings and handles SQL generation, persistence-context management, and dirty checking.

45. BPS-Specific Questions — Inspect Your Codebase

Use these to connect the theory to your actual BPS implementation without assuming how BPS is designed.

Entity mapping

- What are the main BPS entities?
- Which class is annotated with `@Entity` for payment transactions?
- What database table does it map to?
- What is its primary key?
- How is the ID generated: `IDENTITY`, `SEQUENCE`, UUID, or something else?

Relationships

Look for:

```
@OneToOne
@OneToMany
@ManyToOne
@ManyToMany
```

Then determine:

- What does each relationship represent in the business domain?
- Which entity owns the relationship?
- Where is `mappedBy` used?
- Where is `@JoinColumn` used?
- Which database column is the FK?

Fetching

- Are BPS relationships `LAZY` or `EAGER` ?
- Are there repository queries using `JOIN FETCH` ?
- Is `@EntityGraph` used?
- Have you encountered an N+1 query?
- Is `LazyInitializationException` handled through proper transaction/query design?

Persistence behavior

- Where are entities loaded and modified?
- Is `@Transactional` used around the service operation?
- Does Hibernate rely on dirty checking?
- Is `save()` explicitly called after modifying managed entities?
- Is optimistic locking with `@Version` used anywhere?

Most useful exercise

Take **one real BPS payment entity** and trace:

```
Java Entity
↓
@Table
↓
@Id
↓
@Column
↓
Relationships
↓
@JoinColumn / mappedBy
↓
Database table
↓
Repository query
↓
Generated SQL
```

Being able to explain one real entity end-to-end will make JPA/Hibernate questions substantially easier in your interview.